

Part 1

Run	Total Hits	π Approximation	Execution Time (s)
1	32,059,794	2.56478352	1.1993
2	31,446,086	2.51568688	1.1982
3	32,797,974	2.62383792	1.1423
4	32,445,277	2.59562216	1.1655
5	33,482,869	2.67862952	1.2238

Part 2

Run	Single-thread π	Single-thread Time (s)	4-thread AtomicLong π	4-thread Time (s)
1	3.14177296	1.6949	3.14155488	1.1985
2	3.14163448	1.7229	3.14187112	1.1485
3	3.14185160	1.7243	3.14199424	1.1091
4	3.14119952	1.7588	3.14192608	1.1215
5	3.14202480	1.7437	3.14203512	1.1142

Part 3

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread	Efficiency
1	3748.849	1.000×	100.00%
2	2030.160	1.847×	92.33%

4	1008.631	3.717×	92.92%
8	520.350	7.204×	90.06%
16	395.282	9.484×	59.27%
32	387.998	9.662×	30.19%

Q.

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

My CPU has 8 physical cores, so 8 threads already provide a high level of parallelism. Increasing the number of threads to 16 means that multiple software threads must share the available CPU cores and other hardware resources. This creates scheduling, cache, and resource-contention overhead. Therefore, 16 threads cannot provide twice the performance of 8 threads.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized version was not slower than the single-threaded version on my machine. The 4-thread AtomicLong implementation averaged 1.1384 seconds, while the single-threaded implementation averaged 1.7289 seconds, giving approximately a 1.52× speedup. Although AtomicLong introduces synchronization overhead because multiple threads access the same counter, the workload was sufficiently parallelizable that the benefit of using four CPU cores outweighed this overhead. Therefore, the synchronization overhead did not eliminate the performance advantage of parallel execution in my experiment.